

Cloud Computing & déploiement IA

Pr. Mohammed AMEKSA

Enseignant Chercheur à la Faculté des Sciences Semlalia – FSSM

E-mail :

fssm.ameksa@gmail.com

Série TP 2

Axe I. Conteneuriser un modèle IA



Version naïve de l'image

Optimisation mesurée

Multi-étapes et non-root

Axe II. Orchestrer et sécuriser



Orchestration avec Compose

Analyse de vulnérabilités

TP 2 — Conteneuriser un modèle et optimiser son image

Séance 2 : virtualisation, conteneurisation et Docker.



L'image d'inférence produite aujourd'hui compte

Vq être utiliser sur Kubernetes au TP 3

1. Objectifs

À la fin de ce TP, vous saurez :

- ➔ conteneuriser un entraînement puis une inférence de manière **reproductible** ;
- ➔ **mesurer** l'effet de chaque optimisation d'image
- ➔ orchestrer deux conteneurs avec un **volume partagé** via Docker Compose ;
- ➔ analyser les **vulnérabilités** d'une image et en corriger (cet aspect à pour objectif de faire une recherche s'auto-informer)

Avant de commencer

- ➔ Docker installé et fonctionnel (vu au TP 1) : `docker --version` ;
- ➔ Docker Compose : `docker compose version` ;
- ➔ Trivy installé (voici le lien [Trivy](#))

Rappel — Construire une image Docker

Une image est un **empilement de couches** en lecture seule.

Chaque instruction qui modifie le système de fichiers (**RUN**, **COPY**, **ADD**) ajoute une couche.

Un conteneur ajoute au-dessus une **couche inscriptible éphémère**, perdue à sa suppression.

Les instructions d'un Dockerfile

Instruction	Rôle
<code>FROM image:tag</code>	image de base, point de départ de la construction
<code>WORKDIR /app</code>	répertoire de travail des instructions suivantes
<code>COPY src dest</code>	copie des fichiers du contexte de construction dans l'image
<code>RUN commande</code>	exécute une commande pendant la construction (nouvelle couche)
<code>ENV CLE=valeur</code>	variable d'environnement, disponible à l'exécution
<code>EXPOSE 8000</code>	documente le port utilisé par le conteneur
<code>USER appuser</code>	utilisateur qui exécute la suite et le conteneur
<code>HEALTHCHECK</code>	commande de vérification périodique de l'état du service
<code>CMD / ENTRYPOINT</code>	commande lancée au démarrage du conteneur

Le cache de construction

Docker réutilise une couche si l'instruction **et** ses entrées (les fichiers copiés) n'ont pas changé.

Dès qu'une couche est invalidée, **toutes les couches suivantes** sont reconstruites : l'ordre des instructions a donc un effet direct sur la durée de construction (comme c'est déjà mentionné dans le cours)

Commandes essentielles

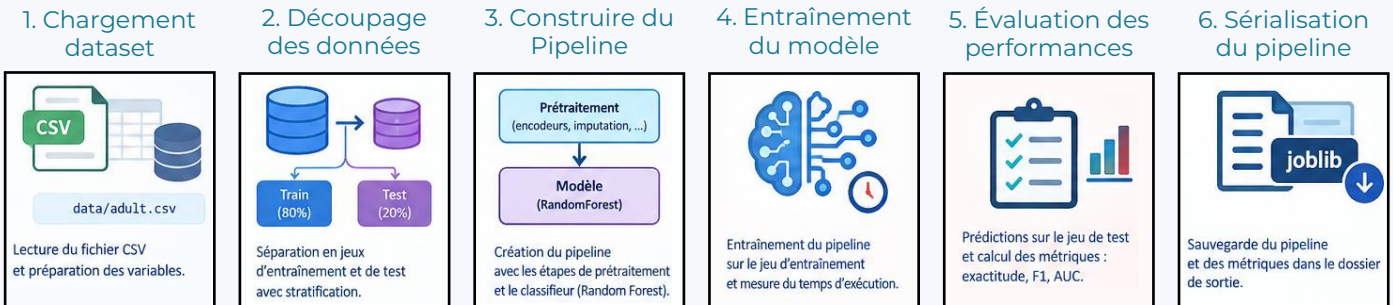
Une fois le Dockerfile est prêt pour construire une image on utilise la commande suivante :

```
docker build -f <fichier> -t nom:tag .
```

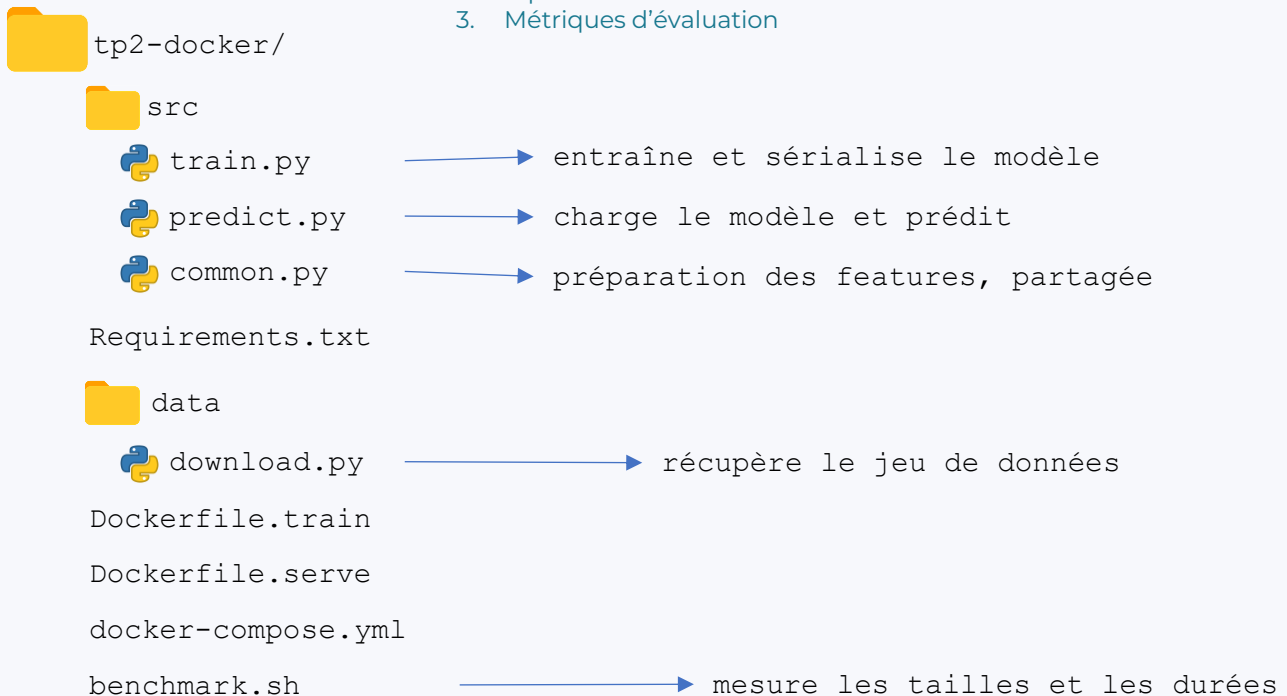
Voir le cours et/ou spreadsheet mentionné pour les autres commandes

2. Le contexte

Vous disposez d'un script d'entraînement (`src/train.py`) qui apprend un classifieur de revenus sur le jeu *Adult Income*, et d'un script d'inférence (`src/predict.py`) qui charge le modèle et prédit sur un lot de données ou via une petite API HTTP (FastAPI).



1. Affichage des métriques (JSON)
2. Pipeline sérialisé
3. Métriques d'évaluation



`src/common.py` est importé **à la fois** par l'entraînement et par l'inférence : c'est ce qui évite l'écart entre la préparation des données à l'entraînement et en production

2.1 Vérifiez d'abord que tout fonctionne hors conteneur

3. Étape 1 — Version naïve

Écrivez `Dockerfile.train` **sans chercher à optimiser** : c'est la version de référence à laquelle vous comparerez toutes les suivantes.

3.1 Contraintes minimales

- ➔ base `python:3.11` ;
- ➔ installer les dépendances depuis `requirements.txt` ;
- ➔ exécuter `python -m src.train` et écrire le modèle dans `/artifacts`.

```
docker build -f Dockerfile.train -t adult-train:naif .
docker run --rm -v $(pwd)/artifacts:/artifacts adult-train:naif
ls -lh artifacts/model.joblib
```

3.2 L'inférence : `Dockerfile.serve`

Faites de même pour `Dockerfile.serve`, qui charge `/artifacts/model.joblib` et expose la prédiction **en ligne de commande** (`python -m src.predict --input ...`). Toujours sans optimisation.

3.3 Relevez immédiatement vos mesures

```
bash benchmark.sh
```

Le script mesure trois valeurs : la **taille** de l'image, la durée de **construction à froid** (`--no-cache`) et la durée de **reconstruction** après modification d'une ligne de `src/predict.py`

Question 1

- ➔ Quelle est la durée de reconstruction après une simple modification du code ?
- ➔ Pourquoi est-elle **presque égale** à la construction à froid ?

4. Étape 2 — Optimisation progressive

Créez `Dockerfile.serve.opt` (partez d'une copie de la version naïve) et appliquez les optimisations **une par une**, en mesurant après chacune

```
bash benchmark.sh opt1 Dockerfile.serve.opt # puis opt2, opt3...
bash benchmark.sh opt Dockerfile.serve.opt # image finale
```

#	Optimisation	Taille	Build à froid	Rebuild après modif.
0	Version naïve (référence)			
1	Base <code>python:3.11-slim</code>			
2	<code>COPY requirements.txt</code> avant <code>COPY src/</code>			
3	<code>.dockerignore</code> complet			
4	<code>pip install --no-cache-dir</code>			
5	Construction multi-étapes			
6	Utilisateur non-root + <code>HEALTHCHECK</code>			

4.1 Repères pour chaque optimisation

#	Ce qu'il faut faire
1	Remplacer <code>python:3.11</code> par <code>python:3.11-slim</code> (Debian allégée). Évitez <code>alpine</code> : avec musl, numpy et scikit-learn doivent être recompilés.
2	Copier <code>requirements.txt</code> seul , installer les dépendances, puis copier <code>src/</code> .
3	Réactiver le <code>.dockerignore</code> : le contexte envoyé au démon Docker ne contient plus que l'utile.
4	Ajouter <code>--no-cache-dir</code> : le cache de pip ne reste plus dans la couche.
5	Une étape <code>builder</code> installe les dépendances ; l'étape finale ne récupère que le résultat avec <code>COPY --from=builder</code> .
6	Créer un utilisateur (<code>useradd</code>), attribuer les fichiers (<code>--chown</code>), puis <code>USER</code> . Ajouter un <code>HEALTHCHECK</code> sur <code>/health</code> et un <code>ENTRYPOINT</code> en forme exec lançant <code>uvicorn src.predict:app</code> .

4.2 Commandes utiles

```
docker images --format "{{.Repository}}:{{.Tag}}\t{{.Size}}"  
docker history adult-serve:opt          # poids de chaque couche  
dive adult-serve:opt                    # exploration interactive des couches
```

Question 2

- ➡ Quelle optimisation apporte le **plus gros gain de taille** ?
- ➡ Laquelle apporte le plus gros gain de **temps de reconstruction** ?
- ➡ Ce ne sont pas les mêmes : expliquez pourquoi.

Question 3

- ➡ Ouvrez **dive** sur votre image optimisée. Quel est son **score d'efficacité** ?
- ➡ Quelle couche gaspille le plus d'espace, et pourquoi ?